Week 6: Researcher Agents

IIMT3688: Advanced AI Applications in Business

Instructor: Xin Tong (xint@hku.hk)

Location: KK 1339 · Spring 2026

Today's Agenda

1. **Why Multiple Agents?** – Limitations of single agents
2. **Agent Specialization** – Researcher, analyst, writer, editor
3. **Multi-Agent System** – Sequential, parallel, hierarchical execution
4. **Research Workflow** – End-to-end pipeline with specialized agents
5. **Agent Handoffs** – Passing context between agents
6. **Sentinel Week 6** – Building the newsroom
7. **Lab 6 Preview** – Context integration challenge

Learning Objectives

- ✅ Understand multi-agent systems and their benefits
- ✅ Design specialized agents for specific tasks
- ✅ Implement agent collaboration patterns
- ✅ Build a research workflow with multiple agents
- ✅ Handle agent handoffs and context passing

Why Multiple Agents?

Single Agent Limitations:

- One perspective, limited specialization
- Sequential processing only
- No parallel work

Multi-Agent Benefits:

- **Specialization**: Each agent excels at specific tasks
- **Parallelism**: Agents work simultaneously
- **Diversity**: Multiple perspectives and approaches
- **Robustness**: If one fails, others continue

Agent Specialization

Different agents for different roles:

Agent	Responsibility
Researcher	Finds and gathers information
Analyst	Analyzes and interprets data
Writer	Creates reports and summaries
Editor	Reviews and improves content
Fact-Checker	Verifies information

Collaboration Patterns Overview

1. **Sequential**: Agent A → Agent B → Agent C
2. **Parallel**: Multiple agents work simultaneously
3. **Hierarchical**: Manager agent coordinates workers
4. **Collaborative**: Agents share and build on each other's work

Technical Concepts

AgentResult Data Class

```python
from dataclasses import dataclass

@dataclass
class AgentResult:
    """Result from an agent execution."""
    agent_name: str
    task: str
    result: str
    metadata: dict = None
```

A standard envelope for all agent outputs.

MultiAgentSystem: Setup

```python
class MultiAgentSystem:
    """System with multiple specialized agents."""

    def __init__(self, agents):
        self.agents = agents  # Dict[str, SimpleAgent]
```

Register agents by name so tasks can reference them.

Sequential Execution (1/2)

```python
    def execute_sequential(self, tasks):
        """Execute tasks in order, passing context forward."""
        results = []
        context = ""

        for task in tasks:
            agent_name = task['agent']
            task_description = task['task']

            if agent_name not in self.agents:
                continue
```

Sequential Execution (2/2)

```
            full_task = (f"{task_description}\n\n"
                         f"Previous Context:\n{context}")

            agent = self.agents[agent_name]
            result = agent.execute(full_task)

            results.append(AgentResult(
                agent_name=agent_name,
                task=task_description,
                result=result
            ))
            context += f"\n{agent_name} result: {result}\n"

        return results
```

Each agent receives accumulated context from previous agents.

Parallel Execution

```
    def execute_parallel(self, tasks):
        """Execute independent tasks simultaneously."""
        import concurrent.futures

        def run_one(task):
            agent = self.agents.get(task['agent'])
            if not agent:
                return None
            result = agent.execute(task['task'])
            return AgentResult(task['agent'], task['task'], result)

        with concurrent.futures.ThreadPoolExecutor() as pool:
            futures = [pool.submit(run_one, t) for t in tasks]
            return [f.result() for f in
                    concurrent.futures.as_completed(futures)
                    if f.result()]
```

Specialized Research Agents

ResearcherAgent

```python
class ResearcherAgent(SimpleAgent):
    def __init__(self, client, search_tool):
        config = AgentConfig(
            role="Research Specialist",
            goal="Find comprehensive information on given topics",
            backstory="Expert researcher with access to search tools."
        )
        super().__init__(config, client)
        self.search_tool = search_tool
```

Researcher: Topic Research

```
    def research_topic(self, topic, depth="standard"):
        search_results = self.search_tool.execute(
            query=f"Information about {topic}")

        prompt = f"""Research Topic: {topic}

Search Results:
{search_results}

Provide a summary including:
1. Key facts   2. Current developments
3. Important context   4. Related topics
5. Sources and references"""

        summary = self.execute(prompt)
        return {"topic": topic, "summary": summary,
                "sources": search_results, "depth": depth}
```

AnalystAgent

```
class AnalystAgent(SimpleAgent):
    def __init__(self, client):
        config = AgentConfig(
            role="Data Analyst",
            goal="Analyze information and extract insights",
            backstory="Experienced analyst skilled at identifying "
                      "patterns, trends, and implications."
        )
        super().__init__(config, client)

    def analyze_information(self, information, focus="general"):
        prompt = f"""Analyze the following information:
{information}
Focus: {focus}
Provide: 1. Key insights  2. Patterns  3. Implications
4. Recommendations  5. Risks or concerns"""

        return {"analysis": self.execute(prompt), "focus": focus}
```

WriterAgent

```
class WriterAgent(SimpleAgent):
    def __init__(self, client):
        config = AgentConfig(
            role="Technical Writer",
            goal="Create clear, comprehensive reports",
            backstory="Expert technical writer who creates "
                      "well-structured reports from research."
        )
        super().__init__(config, client)

    def write_report(self, research, analysis, fmt="executive"):
        prompt = f"""Create a {fmt} report based on:
Research: {research}
Analysis: {analysis}
Structure: 1. Executive Summary  2. Key Findings
3. Analysis and Insights  4. Recommendations  5. Conclusion"""

        return self.execute(prompt)
```

Research Workflow

End-to-End Pipeline

```
class ResearchWorkflow:
    def __init__(self, client, search_tool):
        self.researcher = ResearcherAgent(client, search_tool)
        self.analyst = AnalystAgent(client)
        self.writer = WriterAgent(client)

    def conduct_research(self, topic):
        # Step 1: Research
        research = self.researcher.research_topic(topic)
        # Step 2: Analyze
        analysis = self.analyst.analyze_information(
            research['summary'], focus="comprehensive")
        # Step 3: Write report
        report = self.writer.write_report(
            research['summary'], analysis['analysis'])

        return {"topic": topic, "research": research,
                "analysis": analysis, "report": report}
```

Batch Research

```python
    def batch_research(self, topics):
        """Research multiple topics sequentially."""
        results = []
        for topic in topics:
            result = self.conduct_research(topic)
            results.append(result)
            time.sleep(1)  # Rate limiting
        return results
```

Process topics one at a time with delays to respect API limits.

Agent Handoff Patterns

HandoffManager

```
class HandoffManager:
    def __init__(self, agents):
        self.agents = agents

    def handoff(self, from_agent, to_agent, context, task):
        if to_agent not in self.agents:
            return f"Error: Agent {to_agent} not found"

        prompt = f"""Previous Agent ({from_agent}) Context:
{context}

Your Task: {task}
Continue the work based on the context provided."""

        return self.agents[to_agent].execute(prompt)
```

Collaborative Work

```
    def collaborative_work(self, agents, shared_context,
                           individual_tasks):
        """Multiple agents work on shared context."""
        results = {}

        for agent_name in agents:
            if agent_name not in self.agents:
                continue
            task = individual_tasks.get(agent_name, "Review")
            full_task = f"""Shared Context:
{shared_context}

Your Specific Task: {task}"""

            results[agent_name] = self.agents[agent_name].execute(
                full_task)

        return results
```

The Newsroom Pattern

NewsroomSystem

```python
class NewsroomSystem:
    def __init__(self, client, search_tool):
        self.reporter = ResearcherAgent(client, search_tool)
        self.editor = SimpleAgent(
            AgentConfig(role="Editor",
                goal="Review and improve content",
                backstory="Experienced editor ensuring accuracy."),
            client)
        self.fact_checker = SimpleAgent(
            AgentConfig(role="Fact Checker",
                goal="Verify facts and claims",
                backstory="Meticulous about accuracy."),
            client)
```

Processing a News Story

```python
def process_news_story(self, topic):
    # Reporter researches
    research = self.reporter.research_topic(topic)

    # Fact checker verifies
    fact_check = self.fact_checker.execute(
        f"Fact-check: {research['summary']}")

    # Editor reviews and improves
    edited = self.editor.execute(
        f"Edit and improve:\n{research['summary']}\n\n"
        f"Fact-check notes:\n{fact_check}")

    return {"topic": topic, "research": research,
            "fact_check": fact_check, "final_story": edited}
```

The Sentinel Case

Week 6: Newsroom

This week, you'll build Sentinel's newsroom — a team of specialized agents:

- Create researcher, analyst, and writer agents
- Implement collaboration workflow
- Handle agent handoffs and context passing
- Coordinate sequential and parallel execution

Business Context: As a manager, you realize one agent isn't enough. You need specialists — researchers, analysts, writers. Your newsroom becomes a team.

Sentinel Challenges

- **Coordination** — How do agents share work?
- **Context passing** — Agents need relevant information from upstream
- **Parallel vs. sequential** — Which workflow fits which task?
- **Failure handling** — One agent's failure shouldn't break the system

Collaboration Patterns Deep Dive

Pattern 1: Sequential Pipeline

Use Case: Tasks must happen in order

```
Agent A → Agent B → Agent C
```

Example: Research → Analysis → Writing

```python
def sequential_workflow(input_data):
    research = researcher.execute(input_data)
    analysis = analyst.execute(research)
    report = writer.execute(analysis)
    return report
```

Pattern 2: Parallel Processing

Use Case: Tasks are independent

```
import concurrent.futures

def parallel_workflow(topics):
    with concurrent.futures.ThreadPoolExecutor() as executor:
        futures = {
            executor.submit(researcher.execute, t): t
            for t in topics
        }
        results = {}
        for future in concurrent.futures.as_completed(futures):
            topic = futures[future]
            results[topic] = future.result()
    return combine_results(results)
```

Pattern 3: Hierarchical Coordination

Use Case: A manager coordinates workers

```
      Manager
         ↓
    ┌────┴────┐
Worker1   Worker2
```

```python
class ManagerAgent:
    def coordinate(self, task):
        if task.type == "research":
            return self.researcher.execute(task)
        elif task.type == "analysis":
            return self.analyst.execute(task)
        else:
            return self.generalist.execute(task)
```

Pattern 4: Consensus Building

Use Case: Multiple perspectives needed

```
Agent A ─┐
Agent B ─┼──→ Consensus
Agent C ─┘
```

Multiple analysts review the same data independently. A final step reconciles their outputs into agreed findings.

Performance Optimization

Caching Results

```
from functools import lru_cache

@lru_cache(maxsize=100)
def expensive_analysis(article_id):
    # Expensive API call happens once per article
    return result
```

Avoid redundant API calls for the same input.

Batch Processing

```python
def batch_process(articles, batch_size=10):
    results = []
    for i in range(0, len(articles), batch_size):
        batch = articles[i:i+batch_size]
        batch_results = process_batch(batch)
        results.extend(batch_results)
    return results
```

Group work into batches to manage throughput and costs.

Async Processing

```python
import asyncio

async def async_workflow(tasks):
    results = await asyncio.gather(*[
        agent.execute_async(task) for task in tasks
    ])
    return results
```

Use `asyncio` when I/O-bound tasks (API calls) dominate runtime.